

Vue.js also offers a reactive utility to mark reactivity. There are slight differences between the two, where `ref` can be used to track primitives and objects, and `reactive` can only be initialized with an object. In general, you can opt for code consistency by choosing `ref` over `reactive`. The only downside is that you have to access the value of the reactive item by a `.value` property in the script block. When using the variable in the template block, it is automatically unwrapped by Vue.js. It's therefore a small concession for being able to use `ref` consistently.

=====

We're now perfectly capable of showing variables in the template. There are cases, however, when you need more advanced expressions, such as, in our example, the requirement of sorting the list. For variables that have no side effects and include reactive data, you can use the Vue.js `computed` function.

=====

The List Item code:

```
<script setup lang="ts">
  defineProps<{
    isChecked?: boolean | false
 }>()
</script>

<template>
  <label :class="{ 'checked': isChecked}">
    <input type="checkbox" :checked="isChecked" />
    <slot></slot>
  </label>
</template>
```

=====

The Todo List with directives and event handling:

```
<script setup lang="ts">
  import { ref } from 'vue';
  import type { Ref } from 'vue';
  import ListItem from './ListItem.vue';

  type Item = {
    title: string,
```

```

        checked?: boolean
    }

    const listItems: Ref<Item[]> = ref([
        { title: 'Make a todo list app', checked: true },
        { title: 'Predict the weather', checked: false },
        { title: 'Play some tunes', checked: false },
        { title: 'Let\'s get cooking', checked: false },
        { title: 'Pump some iron', checked: false },
        { title: 'Track my expenses', checked: false },
        { title: 'Organize a game night', checked: false },
        { title: 'Learn a new language', checked: false },
        { title: 'Publish my work' }
    ])

    const toggleItemChecked = (item: Item): void => {
        item.checked = !item.checked
    }

    const findItemInList = (item: Item): Item | undefined => {
        return listItems.value.find((itemInList) => itemInList.title ===
item.title)
    }

    const updateItem = (item: Item): void => {
        const updatedItem = findItemInList(item);
        if(updatedItem) {
            toggleItemChecked(updatedItem)
        }
    }
</script>

<template>
    <ul>
        <li :key='key' v-for='(item, key) in listItems'>
            <ListItem :is-checked='item.checked' v-
on:click.prevent='updateItem(item)'>{{ item.title }}</ListItem>
        </li>
    </ul>
</template>

```

=====

Typically, you would use **computed** for filtering data, formatting expressions, displaying calculations, or Boolean conditionals.

=====

For variables **that have no side effects** and include reactive data, you can use the Vue.js computed function.

=====

The **computed function is very similar to ref**, in the sense that it follows the same reactivity in updating the DOM when the value changes, and you can even access the value using the `.value` property in the script block! The main difference is that a **computed value is cached** and only updates when one of the inputs changes.

=====

These two functions interface with browsers' abilities to store a string of data, so we need to stringify and parse that object. We're storing the data on the **list-items key**.

To retrieve the data when the component gets loaded. There's a function for it, called **onMounted**, and it is part of the **Vue.js core**, so we can import it similarly to the `ref` and `computed` functions.

The **onMounted function** is what we call a life cycle hook. They are functions that get called at certain points in the life cycle of a component. The main life cycle events are triggered when a component **gets mounted** (or before), **gets updated** (or before), **gets unmounted** (or before), and **gives an error**.

=====

For now, we'll store the API key in a file at the root of your project called `.env`, with the following contents:

```
VITE_APP_WEATHER_API_KEY=Replace this with the key
```

The `VITE_APP_` prefix makes sure that Vite automatically exposes the variable to the application.

Note

With a client-based web application, the key will be exposed by default, since it will be attached to API calls that you can inspect via your browser's network requests. For our purposes, this is fine. In a production-like environment, you would likely proxy the request via your own backend to obfuscate any secrets from the public.

=====

To fetch data from a specific URL:

```
<script setup lang="ts">
import { ref, onMounted } from 'vue'
import type { Ref } from 'vue'

type WeatherData = {
  location: {
    localtime: Date;
    name: string;
    region: string;
  };
  current: {
    temp_c: number;
    temp_f: number;
    precip_mm: number;
    condition: {
      text: string;
      icon: string;
    };
    wind_degree: number;
    wind_kph: number;
    wind_mph: number;
  };
};

type Coords = { latitude: number; longitude: number }
interface Props {
  coords: Coords;
}
```

```

const props = defineProps<Props>()
const data: Ref<WeatherData | undefined> = ref()

const fetchWeather = async (coords: Coords): Promise<WeatherData> => {
  const { latitude, longitude } = coords;
  const q = `${latitude},${longitude}`
  const res = await fetch(
    `https://api.weatherapi.com/v1/current.json?key=${
      import.meta.env.VITE_APP_WEATHER_API_KEY
    }&q=${q}`)

  const data = await res && res.json()
  return data
}

onMounted(async () => {
  const { latitude, longitude } = props.coords
  const weatherResponse = await fetchWeather({latitude, longitude})
  data.value = weatherResponse
})
</script>

<template>
  <div>
    <article v-if="data && data.current">
      {{ data.current }}
    </article>
    <div v-else>Loading...</div>
  </div>
</template>

```

And to use the component with the parameters:

```

<script lang="ts" setup>
import { ref, onMounted } from 'vue'
import type { Ref } from 'vue'
import WeatherReport from './WeatherReport.vue';

type Geolocation = {
  latitude: number;
  longitude: number;
}

```

```

const coords: Ref<Geolocation|undefined> = ref()
const geolocationBlockedByUser: Ref<boolean> = ref(false)
const getGeolocation = async (): Promise<void> => {
    await navigator.geolocation.getCurrentPosition(
        async (position: { coords: Geolocation }) => {
            coords.value = position.coords
        },
        (error: {message: string}) => {
            geolocationBlockedByUser.value = true;
            alert(`The error message is: ${error.message}`)
        })
    });
}

onMounted(async () =>{
    await getGeolocation()
})
</script>

<template>
    <div v-if="coords && !geolocationBlockedByUser">
        Latitude: {{ coords.latitude}} & Longitude: {{ coords.longitude }}
        <WeatherReport :coords="coords" />
    </div>

    <div v-if="geolocationBlockedByUser">User denied access</div>
</template>
=====

const formatDate = (dateString: Date): string => {
    const date = new Date(dateString)
    return new Intl.DateTimeFormat("defaule",{
        "dateStyle": "long",
        "timeStyle": "short",
    }).format(date)
}
=====

```

To use style with the computed-function:

```
<script lang="ts" setup>
import { computed } from 'vue'

interface Props {
  degrees: number
}
const props = defineProps<Props>()
const windStyle = computed(() => ({ "transform":
"rotate("+props.degrees+"deg)"}))
</script>

<template>
  <span class="inline-block" :style="windStyle" aria-hidden="true">
    <!-- sr for screen readers -->
    ⬇ <span class="sr-only">Wind Direction {{ degrees }} degrees</span>
  </span>
</template>
```

```
import { describe, it, expect } from 'vitest'
import { shallowMount } from '@vue/test-utils'
import App from '../App.vue'
import GetLocation from '@components/GetLocation.vue'

describe('App', () => {
  it('renders the GetLocation component', ():void => {
    const wrapper = shallowMount(App)
    expect(wrapper.findComponent(GetLocation).exists()).toBe(true)
  })
})
```

To run the unit test: **npm run test:unit**

A **(shallow) mounted component** will behave just as it would when being rendered by a browser, but we run this in isolation. With Vue Test Utils, you can either **mount** or **shallowMount** a component, with the difference being that a mount also attempts to render any children. A **shallowMount function** creates stubs for any children, which reduces side effects and focuses directly on the component itself.

=====

We should always use shallowMount unless we need to assert specific parent-child behavior.

=====

Open the `vite.config.ts` file and look for the `test` property. If we add a new property called `globals` with the value `true` as a child of the `test` property, you should end up with something similar to this:

```
export default defineConfig({
  plugins: [vue()],
  test: {
    globals: true,
    environment: "jsdom",
  }
})
```

Now we can use the functions, but our IDE is not aware of those functions, so we need to configure some additional settings. First, we need to install some types by running the following command in the CLI:

```
npm i --save-dev @types/jest
```

=====

We'll update the `tsconfig.vitest.json` file with the following addition to `types`:

```
{
  "extends": "./tsconfig.app.json",
  "exclude": [],
  "compilerOptions": {
    "composite": true,
    "lib": [],
    "types": ["node", "jsdom", "jest"]
  }
}
```

=====


```

import { shallowMount } from "@vue/test-utils"
import WindDirection from "@components/WindDirection.vue"

describe("WindDirection", () => {
  it("renders without crashing", ():void => {
    const wrapper = shallowMount(WindDirection, {
      props: { degrees: 90 }
    })
    expect(wrapper).toBeTruthy()
  })

  it("renders the indicators with the correct wind direction", () => {
    const wrapper = shallowMount(WindDirection, {
      props: {
        degrees: 90
      }
    })
    const direction = wrapper.find("[data-testid=direction]")
    expect(direction.attributes("style")).toContain("rotate(90deg)")
    expect(direction.html()).toContain("⬇")
  })
})

=====

it("renders the correct wind direction for screen readers", () => {
  const wrapper = shallowMount(WindDirection, {
    props: {
      degrees: 270
    }
  })
  const srOnly = wrapper.find("[data-testid=direction-sr]")
  expect(srOnly.classes()).toContain('sr-only')
  expect(srOnly.html()).toContain('Wind Direction 270 degrees')
})

=====

it("should return data", async () => {
  const mockData = {
    location: {
      localtime: new Date(),
      name: 'Eindhoven',
      region: 'North Brabant',
      country: 'Netherlands',
      lat: 52.37,
      lon: 4.9
    }
  }

```

```

    },
    current: {
      temp_c: 10,
      temp_f: 50,
      humidity: 50,
      precip_in: 0.1,
      precip_mm: 2.5,
      condition: {
        text: 'Sunny',
        icon: 'https://via.placeholder.com/150'
      },
      wind_degree: 180,
      wind_dir: 'S',
      wind_kph: 10,
      wind_mph: 6.2
    }
  }
}

global.fetch = vi.fn(() => Promise.resolve({
  json: () => Promise.resolve(mockData)
}))) as any

const wrapper = await mount(WeatherReport, {
  props: {
    coords: {
      latitude: 0,
      longitude: 0
    }
  }
})

await flushPromises();

expect(wrapper.text()).toContain(mockData.current.condition.text)
expect(wrapper.text()).toContain(mockData.current.temp_c)
expect(wrapper.text()).toContain(mockData.location.name)
expect(wrapper.text()).toContain(mockData.location.region)
expect(wrapper.text()).toContain(mockData.current.wind_kph)
// expect(wrapper.text()).toContain(mockData.current.wind_degree)
})

```